

Laboratório — Contrato Inicial da API de Chamados

Cenário

Uma empresa está iniciando o desenvolvimento de uma aplicação web para gestão de chamados de suporte interno. A equipe de front-end precisa construir telas para listar, registrar e acompanhar chamados. Em paralelo, a equipe de back-end implementará a API e, posteriormente, a persistência de dados.

Para evitar que cada equipe faça suposições diferentes, é necessário criar um contrato inicial da API. O contrato será usado como referência comum durante a implementação.

Objetivo

Elaborar uma especificação inicial, em Markdown ou YAML, para o recurso chamados. A especificação deve permitir que outra equipe compreenda como consultar, criar, consultar individualmente, atualizar e remover chamados sem precisar de explicações informais.

Duração total

45 minutos

Organização

- Formar equipes de 3 a 5 pessoas.
- Definir uma pessoa para registrar o contrato e uma pessoa para apresentar uma decisão técnica no fechamento.
- Utilizar editor de texto, editor Markdown ou arquivo YAML local.
- Não é necessário implementar código, criar conta em serviços externos ou utilizar provedor de nuvem.
- Na ausência de computadores, produzir o contrato em papel ou quadro e registrar uma fotografia para posterior transcrição.

Recursos

- Editor de texto simples, editor Markdown ou editor de código.
- Modelo de contrato apresentado nesta atividade.
- Material de apoio sobre HTTP, REST, JSON e OpenAPI.
- Repositório do projeto integrador, caso a equipe já tenha um.

Distribuição do tempo

Etapa	Duração	Resultado esperado
1. Compreender o recurso	10 min	Finalidade e atributos principais definidos
2. Modelar endpoints	15 min	URLs, métodos e parâmetros definidos
3. Escrever mensagens	15 min	Exemplos de requisição, resposta e erros
4. Revisar entre pares	5 min	Ajustes e dúvidas registradas
Total	45 min	Contrato inicial revisado

Passo a passo

1. Definir a finalidade e os atributos do recurso — 10 minutos

Registrem a finalidade do recurso chamados em uma frase. Exemplo: “Um chamado registra uma solicitação de suporte feita por uma pessoa usuária.”

Em seguida, definam atributos mínimos. Uma sugestão inicial é:

Atributo	Tipo sugerido	Obrigatório na criação?	Exemplo
id	número inteiro	Não	42
titulo	texto	Sim	"Tela sem acesso"
descricao	texto	Sim	"Não consigo acessar a tela de consultas."
prioridade	texto	Sim	"alta"
status	texto	Não	"aberto"

A equipe pode adaptar os atributos, mas deve manter apenas os campos necessários para o primeiro contrato. Definam também valores aceitos para prioridade e status, se essa decisão for tomada.

2. Definir URIs, métodos e parâmetros — 15 minutos

Criem uma tabela com operações orientadas ao recurso. Utilizem nomes no plural e evitem colocar verbos na URI.

Operação	Método	URI	Parâmetros	Status de sucesso previsto
Listar chamados	GET	/chamados	Opcional: status	200
Consultar chamado	GET	/chamados/{id}	id na URI	200
Criar chamado	POST	/chamados	Corpo JSON	201
Atualizar chamado	PATCH	/chamados/{id}	id e corpo JSON	200
Remover chamado	DELETE	/chamados/{id}	id na URI	204

Revisem se cada método representa adequadamente a intenção da operação. Por exemplo, não usem GET /criarChamado; a criação deve ser representada por POST /chamados.

3. Elaborar exemplos de mensagens e respostas — 15 minutos

Escrevam pelo menos um exemplo completo de criação bem-sucedida.

```
POST /chamados
Content-Type: application/json

{
  "titulo": "Tela sem acesso",
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta"
}

HTTP/1.1 201 Created
Content-Type: application/json
Location: /chamados/42

{
  "id": 42,
  "titulo": "Tela sem acesso",
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta",
  "status": "aberto"
}
```

Incluam também dois cenários de erro:

- 1 **Dado inválido:** criação sem o campo titulo, com resposta 400.
- 2 **Recurso inexistente:** consulta de /chamados/9999, com resposta 404.

Exemplo de resposta de erro:

```
{
  "erro": "Dado inválido",
  "detalhes": [
    {
```

```
"campo": "titulo",  
"mensagem": "O título é obrigatório."  
}  
]  
}
```

Definam uma estrutura semelhante para os erros do contrato. A mensagem deve ser compreensível para quem implementará a interface.

4. Revisão por outra equipe — 5 minutos

Troquem o contrato com outra equipe. A equipe revisora deve verificar:

- se as URIs representam recursos, e não ações;
- se os métodos HTTP são coerentes com as operações;
- se os exemplos de JSON são válidos e consistentes;
- se os status de sucesso e erro fazem sentido;
- se seria possível implementar um consumidor da API apenas com base no documento.

Registrem pelo menos um ajuste realizado ou uma dúvida pendente após a revisão.

Modelo de entregável

```
# Contrato inicial – API de Chamados

## Recurso

- Nome: `chamados`
- Finalidade: registrar e acompanhar solicitações de suporte.

## Formato de dados

- Requisições e respostas: JSON

## Atributos

| Campo | Tipo | Descrição |
|---|---|---|
| id | número | Identificador do chamado |
| titulo | texto | Resumo do problema |
| descricao | texto | Detalhamento do problema |
| prioridade | texto | Nível de prioridade |
| status | texto | Situação do chamado |

## Endpoints

| Método | URI | Finalidade | Status previstos |
|---|---|---|---|
| GET | /chamados | Lista chamados | 200 |
| GET | /chamados/{id} | Consulta um chamado | 200, 404 |
| POST | /chamados | Cria um chamado | 201, 400 |
| PATCH | /chamados/{id} | Atualiza parcialmente um chamado | 200, 400, 404 |
| DELETE | /chamados/{id} | Remove um chamado | 204, 404 |

## Exemplos

[Inserir exemplos de requisição e resposta]

## Decisões e dúvidas pendentes

- [Registrar decisões tomadas]
- [Registrar dúvidas que precisam de validação]
```

Entregável

Um arquivo em Markdown ou YAML, preferencialmente incluído no repositório do projeto, contendo:

- finalidade e atributos do recurso chamados;
- endpoints, URIs e métodos HTTP;
- parâmetros de URI ou consulta, quando aplicáveis;
- formato JSON para mensagens;
- exemplo de requisição de criação;
- exemplo de resposta de sucesso;
- exemplo de erro 400 por dado inválido;

- exemplo de erro 404 por recurso inexistente;
- códigos de status previstos;
- decisões e dúvidas pendentes.

Critérios de avaliação

Critério	Evidência esperada
Modelagem do recurso	Finalidade e atributos compreensíveis
Consistência REST	URIs orientadas a recursos e métodos adequados
Clareza do contrato	Outra equipe consegue interpretar as operações
Uso de HTTP	Códigos de status coerentes com sucesso e erro
Qualidade dos exemplos	JSON válido, campos consistentes e mensagens compreensíveis
Colaboração	Revisão por pares e registro de ajuste ou dúvida

Checklist final

- ☐ O recurso possui nome, finalidade e atributos definidos.
- ☐ As coleções usam URI no plural, como /chamados.
- ☐ Um item individual é identificado por URI, como /chamados/{id}.
- ☐ Cada operação possui método HTTP coerente.
- ☐ Há exemplo de POST com corpo em JSON.
- ☐ Há exemplo de resposta 201 para criação bem-sucedida.
- ☐ Há cenário de erro 400 para dado inválido.
- ☐ Há cenário de erro 404 para recurso inexistente.
- ☐ Os exemplos usam os mesmos nomes de campos.
- ☐ Decisões e dúvidas pendentes foram registradas.
- ☐ Outra equipe revisou o contrato.